TECHNICAL REPORT / BENCHMARK SNAPSHOT

EVM-Tailored Post-Quantum Multisignature Verification

Modifications, measured gains, and comparative evidence

Project implementation report • 28 October 2026

Executive findings

The implemented Solidity wallet verifies three experimental signature profiles directly on-chain: K-WOTS8-Merkle, an ML-DSA-44-like Keccak counter profile, and a Falcon-512-like Keccak counter profile. The latter two retain their respective lattice dimensions and norm checks, but substitute specified hash constructions. They do not constitute standard ML-DSA or Falcon/FN-DSA.

Operation	Transaction gas	Local reduction
K-WOTS8, height 3	155,554	3.8% vs. base 16
ML-DSA-like Keccak	775,146	13.5% vs. pinned Keccak
Falcon-like Keccak32	258,600	82.6% vs. pinned Keccak
Mixed 3-of-3 wallet execution	1,240,208	No wallet baseline

Table 1: Recorded local samples. Transaction gas includes base cost, calldata, and applicable transaction accounting. Setup is separate.

What changed. The principal additions are a four-row packed ML matrix layout, a directly packed challenge sampler, native-Keccak Falcon hash-to-point with unbiased 32-bit draws and an eight-output common path, a base-8 hash-chain profile, and a wallet with immutable key data and persistent one-time-key retirement tracking. Existing lattice kernels are reused and attributed.

What the strongest gain means. The Falcon result is 5.73 times cheaper in total transaction gas than the particular locally reproduced Keccak path. A newer published optimized SHAKE-based Falcon verifier is a stronger comparison point; our custom verifier's execution gas is approximately 50.3% below that published figure, without an identical-workload reproduction of that newer result.

The cost of the ML improvement. The expanded key costs 8,779,908 gas to deploy. Its extra cost over the local baseline's key is recovered after approximately 33 verifications, considering key deployment and per-verification savings alone.

Scope of the performance claim. The evidence establishes specific measured wins. It does not establish that this implementation beats every available implementation, preserves standard interoperability, or has equivalent cryptographic assurance. The hash-based comparison has no demonstrated universal lead. All custom profiles remain unaudited.

1 System, provenance, and measurement boundary

1.1 Authorization architecture

An owner registers a root or an immutable expanded verification key and an algorithm identifier. Owners sign a common transaction digest off-chain. A relayer supplies an ordered approval set to the wallet, which verifies each approval, checks the total threshold and optional algorithm minima, and executes the authorized calls atomically. No external signature verdict, signature oracle, SNARK verifier, or chain-specific PQ precompile is used.

Public-key expansion is performed offline. The authorized object is the expanded verification key itself. Registration checks its representation and arithmetic bounds; it does not prove derivation from a compact standard key. This distinction is part of the design's trust boundary, not an unreported off-chain verification shortcut.

ID / profile	Verification material	Wire signature
1 / K-WOTS8	32-byte Merkle root	$2916 + 32H$ bytes
2 / ML-like	Two data parts and a descriptor	2,420 bytes
3 / Falcon-like	1,024-byte NTT key in bytecode	1,064 bytes

ML-DSA and Falcon are two constructions in the lattice family; K-WOTS supplies a hash-based family. The wallet is a collection of independent signatures with threshold counting, rather than a signature aggregation protocol.

1.2 Reused work and project contributions

The ML core reuses the pinned ZKNOX packed decoder, hint encoder, and fused NTT kernels. The Falcon core reuses the pinned ZKNOX eight-lane product and signature-norm logic. Reference key generation and signing machinery come from the pinned noble package. These are not claimed as newly invented algorithms. The row-oriented key layout, selected matrix kernel, direct challenge implementation, custom Falcon sampling path, wallet, stateful signer integration, and comparative harness are project additions.

Component	Pinned source / configuration
ETHFALCON	`012e08074e8c1a2b79b046ac8eac740122ec3a73`
ETHDILITHIUM	`f85ca80c446c2d59fcd68b5efe565275599bb1f9`
noble reference	`@noble/post-quantum@0.7.1`
Compiler	solc 0.8.30, via-IR, 1,000,000 optimizer runs
Execution	Local EthereumJS Prague; bytecode targets Cancun

1.3 Two measurements, two evidence classes

Execution gas is the local EVM's gross execution counter. It excludes transaction base and calldata charges. **Transaction gas** is the signed transaction's total gas spent, including calldata, refunds where applicable, and the Prague calldata floor. Subtracting execution gas from transaction gas is not a general decomposition when refunds or a floor apply.

L denotes a locally reproduced observation. **P** denotes an external author-reported number consulted on 28 October 2026. Published numbers are comparison context, not results rerun under this harness.

2 ML-DSA-like profile: hashes and key representation

2.1 Preserved arithmetic and explicit hash substitution

The selected profile retains the ML-DSA-44 core's $q = 8{,}380{,}417$, polynomial dimension 256, four matrix rows and four columns, challenge weight 39, 2,420-byte signature representation, strict z norm checks, and canonical hint parsing. Its Keccak counter profile matches the pinned ETHDilithium variant rather than inventing a smaller lattice parameter set.

Let $K = \text{Keccak256}$. The counter construction used for public verification hashing is

$$s = K(\text{input}), \tag{1}$$

$$B_j = K(s \parallel \text{uint64}_{\text{BE}}(j)), \quad j = 0, 1, \ldots \tag{2}$$

The stream concatenates $B_0, B_1, \ldots$ in byte order. The ML message input uses $m' = \texttt{00} \parallel \texttt{00} \parallel d$, an empty-context encoding of the wallet digest d. Verification reconstructs

$$\mu = \text{CTR}(\text{tr} \parallel m', 64), \tag{3}$$

$$c = \text{SampleInBall}_{\text{CTR}}(\widetilde{c}), \tag{4}$$

$$\widetilde{c}' = \text{CTR}(\mu \parallel \text{Encode}(w_1), 32), \tag{5}$$

and requires $\widetilde{c}' = \widetilde{c}$ after the lattice and hint checks. The 64-byte tr remains SHAKE256 of the original compact public key. The generated reference signer changes these public message/challenge hashes and SampleInBall; its secret-key generation, signing-mask expansion, and matrix expansion are retained. Standard ML-DSA uses SHAKE in these locations, so the selected profile is a distinct experimental scheme.

2.2 Column-major storage, four rows per word

For coefficient index i, precompute the packed column word

$$P_j[i] = \sum_{r=0}^{3} \widehat{A}_{r,j}[i]\, 2^{64r}, \qquad j = 0, 1, 2, 3, \tag{6}$$

and the corresponding packed key term

$$T[i] = \sum_{r=0}^{3} \text{NTT}(2^{13} t_{1,r})[i]\, 2^{64r}. \tag{7}$$

Each stored field is canonical and less than q. The five columns contain $5 \cdot 256 \cdot 32 = 40{,}960$ bytes. They are split across two 20,480-byte data payloads, each with a STOP byte. This keeps each deployed data runtime below 24,576 bytes.

The descriptor payload is

$$\text{part}_0[20] \parallel \text{part}_1[20] \parallel \text{tr}[64].$$

It has 104 bytes, or 105 bytes including STOP. Canonical field checks and fixed lengths establish the bounds needed by the packed arithmetic. Content identity is $K(\text{tr} \parallel \text{expandedColumns})$, so copying the same expanded credential to another address does not create another vote.

3 ML-DSA-like profile: matrix and challenge kernels

3.1 One EVM multiplication supplies four row products

The transformed signature coefficient $\widehat{z}_j[i]$ is a scalar. Its product with $P_j[i]$ computes four row products simultaneously:

$$P_j[i]\widehat{z}_j[i] = \sum_{r=0}^{3} (\widehat{A}_{r,j}[i]\widehat{z}_j[i])2^{64r}. \tag{8}$$

The implementation forms the four-row raw accumulator

$$R[i] = \sum_{r=0}^{3}(q2^{28})2^{64r} + \sum_{j=0}^{3} P_j[i]\widehat{z}_j[i] - T[i]\widehat{c}[i]. \tag{9}$$

The offset prevents subtraction borrow between lanes. Given the registered key bounds and the reused transform's $\widehat{z}, \widehat{c} < 17q$ bound, each raw row coefficient lies in $[0, q2^{30})$. For example,

$$q2^{28} - 17q^2 > 0,$$
$$q2^{28} + 68q^2 < q2^{30} < 2^{64}.$$

Thus neither carry nor borrow changes a neighbouring row. The kernel transposes groups of four output words into the existing inverse-NTT/hint layout. It avoids re-unpacking the matrix on every verification.

Component profile, gas (L)	Reference	Selected
Five forward transforms	196,305	196,302
Matrix multiplication	224,704	123,549
Four inverse transforms	179,269	177,863

Matrix execution falls 45.0%. These component profiles are contextual microbenchmarks, not additive standalone transaction measurements. Reproduce with `npm run profile`.

3.2 Challenge sampling directly into packed memory

The new SampleInBall implementation outputs 64 words, each holding four 64-bit coefficient lanes. It avoids the upstream 256-word intermediate, per-byte PRNG helper calls, and a subsequent packing pass. Coefficient payload storage falls from 8,192 to 2,048 bytes; these figures exclude array headers and other memory allocations.

The implementation preserves the first eight bytes' little-endian sign-bit order, rejection of byte draws above the current position, swap/placement semantics, challenge weight 39, and the same counter stream as the compared Keccak profile. No challenge-norm or hint bound is removed.

Challenge measurement (L)	Reference	Selected
Execution gas	46,984	21,863
Transaction gas	68,560	43,439

The execution reduction is 53.5%. The complete ML verifier's local transaction reduction is 13.5%; memory and key I/O costs mean component percentages do not transfer directly to the complete verifier.

4 Falcon-like profile: sampling and norm accumulation

4.1 Preserved lattice verification

The verifier retains $n = 512$, $q = 12{,}289$, and the squared-norm bound $\beta^2 = 34{,}034{,}726$. With the point polynomial u and public key h, it checks

$$\| \operatorname{center}(u - s_2 h) \|_2^2 + \| \operatorname{center}(s_2) \|_2^2 \leq \beta^2. \tag{10}$$

The eight-lane product and compact s_2 norm logic are reused from pinned ETHFALCON. Verification uses integer arithmetic. The reference signer's floating-point Gaussian sampling machinery is unchanged.

4.2 Domain-separated native Keccak expansion

The input to the counter construction is

$$K(\texttt{PQWALLET/FALCON512/POINT32/v1}) \parallel \text{salt}[40] \parallel d[32].$$

This replaces the SHAKE-based point derivation and changes the scheme's interoperability. It removes software SHAKE permutations from the custom verification path. The EVM's Keccak256 opcode is not a drop-in SHAKE opcode.

4.3 32-bit unbiased draws, eight candidates per word

Interpret successive four-byte stream segments as big-endian uint32 values. With

$$L = \left\lfloor \frac{2^{32}}{q} \right\rfloor q = 4{,}294{,}956{,}344, \tag{11}$$

reject $x \geq L$ and map accepted x to $x \bmod q$. Every residue then has the same number of preimages in the accepted range. Unconditional modulo would introduce bias. The rejection probability per draw is $(2^{32} - L)/2^{32} = 10{,}952/2^{32}$.

On the common path, the output index is eight-aligned, eight outputs remain, and a bit-run detector establishes that no candidate has an all-ones upper 16-bit half. This implies each candidate is below $\texttt{0xffff0000} < L$. One loaded product word supplies the eight complement coefficients, and the kernel accumulates eight squared differences without materializing a 512-element point array. Exceptional blocks perform explicit rejection checks and bounded individual accumulation. A fixture forces a true rejection at draw 471, exercising unaligned and tail handling.

4.4 Registered key and signature layout

The stored key is the full ordinary NTT of h expected by the pinned kernel, with sixteen canonical 16-bit residues per 256-bit word. Its 1,024-byte payload is stored in STOP-prefixed code; the signature is a 40-byte salt plus 1,024 bytes of identically packed s_2. This is a separate wire format from the round-3 compressed signature. It saves repeated public-key transmission while trading a larger signature payload for inexpensive decoding.

5 Hash-based profile: chains, checksum, and state

5.1 Base-8 chains rather than the base-16 comparison

K-WOTS8-Merkle uses 32-byte chain elements, 86 message digits, four checksum digits, and 90 chains of seven steps. For digest integer D,

$$d_i = (D \gg (255 - 3i)) \,\&\, 7, \quad i = 0, \ldots, 85, \tag{12}$$

$$S = \sum_{i=0}^{85} (7 - d_i). \tag{13}$$

The 258-bit base-8 representation has two leading zero bits. The four checksum digits are the high-to-low three-bit fields of S. The first message digit consequently carries one message bit, not three.

Each step uses an addressed 128-byte Keccak input:

$$K(\text{CHAIN} \parallel \text{publicSeed} \parallel ((\ell \ll 32) \mid (i \ll 8) \mid j) \parallel x).$$

Here ℓ is the leaf index, i the chain index, and j the step. Leaf compression binds the public seed, leaf index, and all chain endpoints. Parent hashing binds the level and ordered children. Domains are distinct hashes of the corresponding protocol-role strings.

Property	**Base 16**	**Selected base 8**
Message / checksum digits	64 / 3	86 / 4
Chain elements	67	90
Maximum chain steps	15	7
Signature bytes, height 3	2,276	3,012
Execution gas (**L**)	103,312	85,506
Transaction gas (**L**)	161,764	155,554

The shorter chains save hashes but increase calldata. Base 8 wins this local height-3 transaction sample by 3.8%. Taller proofs, signature values, calldata-floor effects, and network pricing can change the preferred choice. No universal base-8 optimum is claimed.

5.2 One-time-key lifecycle

The wire layout is

$$\text{uint32}_{\text{BE}}(\ell) \parallel \text{publicSeed}[32] \parallel \text{chains}[2880] \parallel \text{siblings}[32H],$$

with $0 \le H \le 20$ and $\ell < 2^H$. The benchmark has $H = 3$, hence only eight leaves per root. A height-16 instance would carry 3,428 signature bytes and support 65,536 leaf indices; no complete height-16 workload is benchmarked.

The wallet remembers a root's monotone next-leaf index across removal and re-registration. The CLI separately locks, persists, fsyncs, and atomically commits leaf allocation before releasing a signature. Failed transactions, abandoned proposals, and crashes can burn off-chain leaves. An on-chain rejection cannot repair two different signatures already disclosed under one leaf. Independent state-file copies and rollback from backups remain unsafe.

This addressed-chain construction preserves a checksum but does not implement WOTS+ masking or standard XMSS. It needs its own cryptographic analysis.

6 Storage, wallet, and tooling modifications

6.1 Immutable data instead of persistent coefficient slots

`KeyData.sol` returns a STOP byte followed by public data. Canonical lengths and prefixes are checked when accepting keys. Data can be read with `EXTCODECOPY`; the wallet stores the compact data address or root, rather than thousands of coefficient SSTORE slots. The two-part ML layout respects the ordinary EIP-170 runtime limit. Key data is public and can be shared by wallets.

6.2 Digest binding and threshold policy

The signed digest is

$$d = K(\mathrm{ABI}(\mathrm{DOMAIN}, \mathrm{chainId}, \mathrm{wallet}, \mathrm{epoch}, \\ \mathrm{nonce}, \mathrm{deadline}, K(\mathrm{ABI}(\mathrm{calls})))).$$

The exact typed fields and ABI encoding are supplied by the contract's `digest` method. The epoch increments on each approved configuration change, invalidating pending authorizations under older owner/policy sets.

Modification	Resulting behavior
Ordered owner IDs	Strictly increasing IDs prevent duplicate counting.
Content-based key identity	Equivalent credentials at different data addresses cannot receive extra votes.
Threshold and algorithm minima	Policies can require a total count plus hash/ML/Falcon minimum counts.
Single execution transaction	No per-proposal or per-approval storage transactions.
Transient guard	Cancun transient storage blocks reentry into execution.
Atomic CALL batches	A failed target reverts the whole batch, nonce, and on-chain leaf update.
Self-call configuration	Owner or policy changes require the current authorization policy.
Persistent root retirement	Removing and re-adding a root does not reset its used-leaf high-water mark.

No ECDSA owner override, unrestricted delegatecall, or administrative upgrade key is added. Relayer identity is not spending authority. EVM call success does not interpret token-specific returned booleans. The wallet does not extend post-quantum security to chain consensus or the relayer's own assets.

6.3 Signer, build, and benchmark tooling

Reference signer adaptations are generated from pinned noble sources with explicit source-anchor checks and retained license notices. Private key files use exclusive creation and mode 0600; public expanded keys are exported separately. The low-level WOTS fixture helper is distinct from persistent allocation in the CLI. The build enforces runtime code size, records hashes of compiled Solidity inputs, and rejects stale build artifacts before tests. Benchmarks execute signed local transactions and validate acceptance before recording results. The report generator reads the recorded JSON snapshot.

7 Locally reproduced results and amortization

7.1 End-to-end observations

All entries below are local observations (**L**). Each row's signature and message are checked before measuring its transaction. Standard and custom hash profiles are intentionally kept distinct.

Verifier / operation	Execution	Transaction
Selected K-WOTS8, $H = 3$	85,506	155,554
Base-16 comparison, $H = 3$	103,312	161,764
Selected ML-like Keccak	714,478	775,146
Pinned packed ETHDilithium	834,881	896,389
Pinned SHAKE ML baseline	1,186,529	1,248,061
Selected Falcon-like Keccak32	221,992	258,600
Pinned ETHFalcon Keccak path	1,428,516	1,482,244
Pinned eight-lane SHAKE Falcon	635,425	688,565
Mixed 3-of-3 wallet execution	1,115,024	1,240,208

The local transaction savings are 13.5% for ML versus the matching Keccak-profile baseline and 82.6% for Falcon versus the particular existing Keccak path. The latter includes a different sampling profile, encoding, and use of existing optimized arithmetic. It is not a claim that all of that reduction is a new arithmetic invention.

7.2 Setup costs belong in the comparison

Deployment	Gas	Runtime bytes
Wallet, sample owner set	2,508,682	6,385
Hash verifier	284,029	1,069
ML verifier	3,823,338	17,641
Falcon verifier	3,725,778	17,248
ML key parts and descriptor	8,779,908	Three data runtimes
Local baseline ML key	4,836,510	Separate baseline blob
Falcon key	281,092	1,025

For ML, the extra key deployment is 3,943,398 gas and the recorded per-verification transaction saving is 121,243 gas. Therefore

$$N_{\text{key}} = \left\lceil \frac{3{,}943{,}398}{121{,}243} \right\rceil = 33. \tag{14}$$

This is a key-only break-even point. A complete lifetime comparison must also include registration validation, verifier deployment, sharing of data or helper contracts, key rotation, signature distribution, and usage count.

8 Position against available implementations

The following external numbers are published observations (**P**), not locally normalized reruns. All gas figures in this table refer to verifier execution, not full transaction cost. A smaller number under another hash profile or security construction is not a same-scheme optimization proof.

Published implementation	Gas	Comparison boundary
ZKNOX packed ETHDilithium [2]	847,709	Existing Keccak profile; our 714,478 is indicatively 15.7% lower.
ZKNOX packed SHAKE ML [2]	1,196,707	Standard hash profile; our substituted profile changes interoperability.
Fireblocks ML-DSA-44 [3]	1,226,311	Standard ML-DSA; stronger published assurance evidence than this project.
ZKNOX newer eight-lane Falcon [1]	447,047	SHAKE-based Falcon; our 221,992 is indicatively 50.3% lower under a custom profile.
Hashmind HMS-1 [4]	96,223 median	Range 74,841–115,167 over 1,200 vectors; heights 10/16 differ from our height 3.
Winternitz WOTS [5]	About 80,000	Different rolling-key account design; below our 85,506 single sample.

Which conclusions survive these distinctions? The selected ML layout improves the locally reproduced Keccak implementation while spending more on key storage. The Falcon custom profile has a substantial measured advantage over its local Keccak baseline and a lower execution figure than the newer published SHAKE result. The hash verifier is competitive but has no demonstrated lead over all other hash-based implementations.

Why the 82.6% headline is narrow. It compares with the 1,482,244-gas local Keccak path, not the newer 447,047-gas published SHAKE path. The latter also uses a different key representation and workload. Reproducing it under identical conditions is still required before presenting a normalized head-to-head percentage.

Assurance is a separate axis. Fireblocks reports standard vectors, fuzzing, mutation checks, and machine-checked arithmetic properties, while remaining unaudited [3]. Our assertion count is not comparable to its number of tests or proved properties. Custom-hash security analysis, formal implementation evidence, ecosystem interoperability, key capacity, and operational maturity prevent a universal superiority claim.

Supported wording. “Under the recorded local configuration, the custom ML and Falcon profiles use less transaction gas than the specified pinned Keccak baselines. External published figures suggest competitive performance, with explicit cryptographic, setup-cost, and workload differences.” A claim of beating every existing implementation is not supported by this evidence.

9 Validation, rejected experiments, and remaining work

9.1 Recorded checks

The local integration run records 37,052 assertions. Many are individual coefficient comparisons; this is not a count of distinct signatures or independent security experiments. The suite checks multiple messages, malformed encodings, signature and public-key coefficient bounds, and the exceptional Falcon rejection path.

Wallet cases cover replay, cross-wallet binding, expired authorization, duplicate or unordered approvals, insufficient algorithm minima, altered signatures, duplicate key contents at distinct addresses, authorized configuration changes, root removal/re-addition, failed-batch rollback, and reentry rejection. CLI tests cover private-file permissions, persistent leaf allocation, concurrent processes, lock contention, and leaf exhaustion.

The arithmetic research model separately exercises 253 scalar transform cases, 100 raw-accumulator inverses, and 10,000 five-lane Montgomery operations. Generated code and twiddle tables are checked for reproducibility. Source freshness checks and actual deployments enforce the configured code-size boundary.

9.2 Experiments deliberately excluded from the selected verifier

The first compact transposed ML key layout was correct and reduced deployed code, but did not produce enough verification savings. A subsequent design packed five forward polynomials in 51-bit lanes with Montgomery $R = 2^{26}$, using the unusually small correction constant 8,380,415. It paired this with four inverse polynomials in 64-bit lanes and generated fused radix-8/8/4 schedules.

These transforms passed scalar comparisons but lost gas to conversion and compiler stack traffic. They remain isolated research artifacts. The selected ML verifier uses the existing fused NTT kernels and the expanded four-row matrix layout. Correctness alone was not used to justify a slower replacement.

9.3 Evidence required for stronger claims

1. Independently analyze the custom hash constructions, chain construction, and cross-domain interactions; do not transfer NIST security categories by parameter resemblance alone.
2. Normalize compiler, fork, ABI framing, public-key preparation, memory warmth, message length, security parameters, and setup accounting across the newest comparable sources.
3. Measure many keys and signatures and report distributions, rejection tails, invalid-input costs, and worst-case bounds where established.
4. Compare hash profiles at equal tree capacity and complete lifecycle cost, including signing-state recovery and key generation.
5. Review actual deployed bytecode, wallet configuration paths, persistent signing state, and operational integrations independently.

No audit, cryptographic security proof, public funded deployment, or exhaustive implementation survey has been completed for this project. Keeping lattice bounds unchanged establishes neither hash-profile equivalence nor a standard-compliant verifier.

10 Reproduction and modification inventory

10.1 Build and run

Requirements are Node.js 24, Python 3, and the lockfile-pinned dependencies. Contracts require Cancun or later; tests use Prague transaction accounting.

```
npm ci
npm run build
npm test
npm run profile
python3 docs/latex/generate_report.py
tectonic docs/latex/pq_multisig_report.tex
```

The LaTeX source is standalone after generation. The PDF content derives from the recorded benchmark JSON, not a new performance run. The generator checks for unresolved numeric placeholders. The source benchmark's SHA-256 is

`41fcd8a409b3da088a2a47b6ab4f1ea68c7a7618beedf3505330086f58dda164`

10.2 Where each modification is implemented

Source / tool	Implemented change
`src/MLDSAKeccak.sol`	Expanded-key descriptor, four-row matrix kernel, directly packed challenge, Keccak-profile verification.
`src/FalconKeccak.sol`	KeccakCTR32 point derivation, eight-candidate common path, exact norm accumulation, immutable-key adapter.
`src/KeccakWOTS.sol`	Base-8 chains, checksum, leaf compression, Merkle path, strict wire shape.
`test/KeccakWOTS16.sol`	Base-16 comparison profile.
`src/KeyData.sol`	STOP-prefixed immutable public-key data.
`src/PQWallet.sol`	Threshold/minimum policy, bound digest, nonce/epoch, transient guard, configuration, root retirement, content de-duplication.
`scripts/keys.mjs`	Canonical expanded-key preparation and Falcon signature conversion.
`scripts/keccak-ctr.mjs`	Specified counter stream for reference signers.
`scripts/prepare-signers.mjs`	Explicit pinned-source hash-profile adaptations.
`scripts/wots-cli.mjs`	Locked, durable pre-disclosure leaf allocation.
`scripts/pq-cli.mjs`	Lattice reference signing and separate public key export.
`scripts/build.mjs`	Pinned compiler settings, runtime-size enforcement, source hashes.
`test/integration.mjs`	Accepted/invalid vectors, wallet cases, signed-transaction benchmark.
`test/cli.mjs`; `test/profile.mjs`	State/concurrency checks; isolated component comparison.
`src/ParallelNTT.sol`; generated companions	Correct but unselected parallel-polynomial transform research.

References and evidence sources

[1] ZKNOX, *ETHFALCON: Implementation of FALCON and its variants for Ethereum*. Published benchmark table consulted 28 October 2026. https://github.com/ZKNoxHQ/ETHFALCON. Local baseline uses pinned commit 012e08074e8c1a2b79b046ac8eac740122ec3a73.

[2] ZKNOX, *ETHDILITHIUM: Implementation of MLDSA and its variants for Ethereum*. Published packed-verifier benchmark table consulted 28 October 2026. https://github.com/ZKNoxHQ/ETHDILITHIUM. Local baseline uses pinned commit f85ca80c446c2d59fcd68b5efe565275599bb1f9.

[3] Fireblocks Labs, *Efficient EVM ML-DSA verification*. Verifier gas boundary and validation evidence consulted 28 October 2026. https://github.com/fireblocks-labs/evm-ml-dsa-verifier. Published figures are not independently reproduced here.

[4] Hashmind, *HMS-1: Hash-based signatures over Keccak-256*, version 1, October 2026, parameters and measured costs. https://www.hashmind.fun/docs/hms-1.

[5] Winternitz, *Architecture*, WOTS verifier and rolling-key account description. Consulted 28 October 2026. https://www.winternitz.app/docs/architecture.

[6] NIST, *FIPS 204: Module-Lattice-Based Digital Signature Standard*, 2024. https://csrc.nist.gov/pubs/fips/204/final. Referenced as the standard from which the custom ML hash profile differs.

[7] P.-A. Fouque et al., *Falcon: Fast-Fourier Lattice-based Compact Signatures over NTRU*, specification v1.2, 2020. https://falcon-sign.info/falcon.pdf.

[8] A. Hulsing et al., *XMSS: eXtended Merkle Signature Scheme*, RFC 8391, 2018. https://www.rfc-editor.org/rfc/rfc8391. The implemented addressed-chain construction is not a standard XMSS profile.

[9] P. Miller et al., *noble-post-quantum*, MIT-licensed reference implementations, npm version 0.7.1. https://github.com/paulmillr/noble-post-quantum.

Local evidence and attribution

The implementation, benchmark snapshot, and build manifest are the primary evidence for project-specific claims:

- `benchmarks/latest.json`: measured transaction and execution gas, deployment gas, assertion count, and compiler/fork metadata.
- `artifacts/build.json`: compiler configuration and compiled-source hashes; per-contract artifacts supply runtime bytecode sizes.
- `docs/PROTOCOL.md`: exact custom hashes, encodings, and trust boundary.
- `docs/RESEARCH.md`: the excluded transform experiment.
- `THIRD_PARTY.md`: upstream commit provenance and retained MIT attribution, including upstream Fireblocks-derived components.

No source's self-reported assurance is treated as an independent audit of this project. External sources describe their own implementations; the local code, data, and limitations determine the claims made in this report.